II CHANNEL CODING

3. Channel Coding

3.1. The Basics

To model information transmission over a noisy channel we use the following figure:

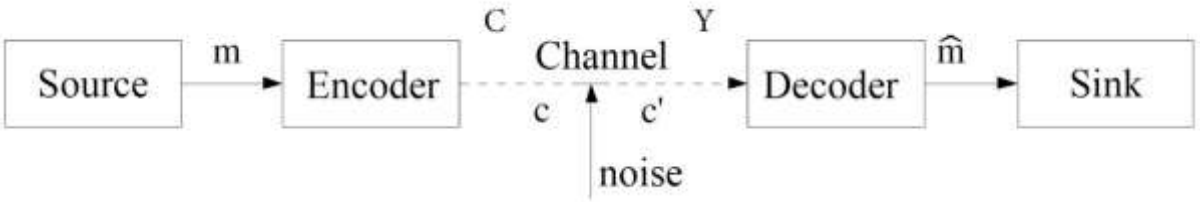


Figure 3.1: *Basic model of information transmission*

In channel coding we suppose that the source and its message m is **already encoded** (compressed). We also suppose that message m is constructed of **binary** symbols. Our task is to transfer this message to the sink (which embeds a source decoder) with as less noise related distortion as possible.

Our basic method to overcome the effects of noise is making m "redundant enough": with **redundancy** we try to make it possible to fix errors with the decoder, i.e. finding the most probable message estimate $\hat{m}$. So in source coding we *reduced* redundancy (in optimal cases and “in general” we even eliminated it), and now we want to *increase* it, based on the properties of the channel and the problem itself. This simple principle has been used with success intuitively in many fields of communication, as an example, consider the code words used in navigation and aviation:

1	UNAONE	A	ALPHA
2	BISSOTWO	B	BRAVO
3	TERRATHREE	C	CHARLIE
4	KARTEFOUR	D	DELTA
5	PANTAFIVE	E	ECHO
6	SOXISIX etc.	F	FOXTROT etc.

The way of increasing redundancy for the most commonly used *block codes* is dividing the continuous input bitstream (consecutive code words of a decodable code) into blocks of length K, and from them computing channel code blocks of length N (N > K).

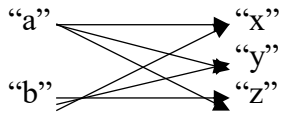
Numbers N and K are called **code parameters**, and a code described by them is said to be a "code of parameters (N, K)” or "(N, K) code", for short.

3.2. Characterizing Channels

We define the **discrete memoryless channel** (DMC), the medium of information transmission, in the following way:

- in every time segment, one input is received at the beginning and one is emitted at the end of the channel (**synchronous behaviour**)
- the input and output symbol sets are not necessarily identical, but both contain a fixed and finite number of elements (**discrete property**)
- if input symbols are statistically independent, then output symbols are also independent (**memoryless source**)

Example 3.1
A channel (transition) graph



The channel has 2 input and 3 output symbols

Let c_i denote the i-th input symbol and y_j the j-th output symbol of the channel. Then, in case of a real channel, we can get the probability of the channel emitting y_j for input c_i (i.e. the conditional probability $p(y_j|c_i)$) by using empirical measurements or with other methods. If we produce $(y_j|c_i)$ for every i and j, then from the information theory point of view we can characterize the channel completely. The probability of receiving y_j in a time instant:

$$p(y_j) = \sum_i p(y_j|c_i)p(c_i)$$

From the $p(y_j|c_i)$ conditional probabilities we can construct the so-called **transition probability matrix P**(Y|C), which contains $p(y_j|c_i)$ in its j-th row and i-th column. The i-th column of P gives us the statistical distribution of the outputs when the input is c_i, thus each column of the matrix must sum up to 1. We can indicate these probabilities on the edges of the channel graph too.

If the channel has no noise, then the input symbol defines uniquely the corresponding output symbol and every $p(y_j|c_i)$ is either 0 or 1. As the amount of noise increases, the probabilities change as well, and it becomes harder and harder to figure out the original input symbol for a given output symbol. The quality of a channel is measured by the average amount of information that can be transferred from the input to the output. If we consider the input and the output as two sources, then this is nothing else but their **mutual information**.

Example 3.2
Let the previous channel's transition probability matrix be

$$\mathbf{P}(\mathrm{Y}|\mathrm{C}) = \begin{bmatrix} 0.7 & 0.1 \\ 0.2 & 0.2 \\ 0.1 & 0.7 \end{bmatrix}$$

Suppose that P(C) = {0.5, 0.5}. *What is the entropy of the output, of the input and what is their mutual information?*
From the definition: H(C) = 1 bit, H(Y) = 1.51 bit, I(C, Y) = 0.36 bits.
Note that because of $p(y_1|c_0) = p(y_1|c_1)$ *the output symbol "*b*" contains no information about* C*, regardless of* C*'s probability distribution.*

The example shows a case when only 0.36 bits of the initial average information content of 1 bit at the input has passed through to the output, as a result of the noise in the channel. Yet at

the output we measure an average information content of 1.51 bits (this excess information – like in the case of town gossip – conveys information about the channel itself).

If we consider not just one specific input distribution, we can define the **channel capacity C** as *the maximum average amount of information that can be sent per channel use* for any input distribution, i.e.

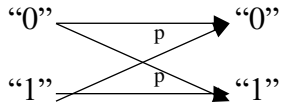
$$C = \max_{P(C)} I(C,Y)$$

Finding the channel capacity through theoretical methods is a difficult task, however, there exists a numerical algorithm that can provide a result of arbitrary accuracy (this is the Arimoto-Blahut algorithm). Further information can be found in the corresponding literature.

Example 3.3

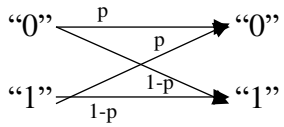
Some important channels and their capacity

***Binary symmetric channel**, BSC: the probability of error is p*



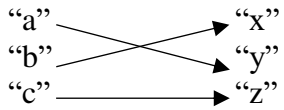
C = 1-H(p 1-p)

0-transmission channel

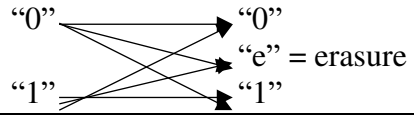


C = 1-H(p, 1-p)

Noiseless channel



Binary erasure channel



We should take a look at the capacity of BSC (see Figure 3.2). We can see that a channel with p = 0 is just as noiseless as one with p = 1. The latter is also often called as the *inverter* channel. Just like for people who *always* give us wrong advice, if we do the opposite of what they say, we can be sure to do the right thing.

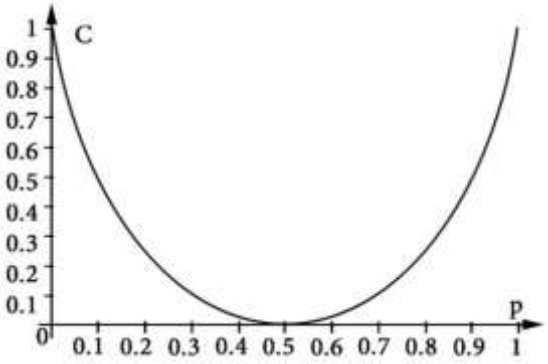


Figure 3.2: *The channel capacity of a binary symmetric channel, as a function of error probability p*

Channel capacity is similar to the width of a road: it depends on technical construction, not on the information transmitted – just like as a road's width is independent of the traffic it has. As

we defined capacity with the maximal mutual information, it is clear that in *general*, information of higher amounts than the capacity cannot be transmitted through the channel. However, for some specific cases this might not be true, e.g. it can happen that an original message – without any channel coding – passes through with no distortion, despite the presence of noise.

3.3. The Binary Communication Problem

Let us return to our channel model and take a look at the physical properties of the channel. Figure 3.3 shows the inner model of a discrete noisy channel, between the encoder and the decoder.

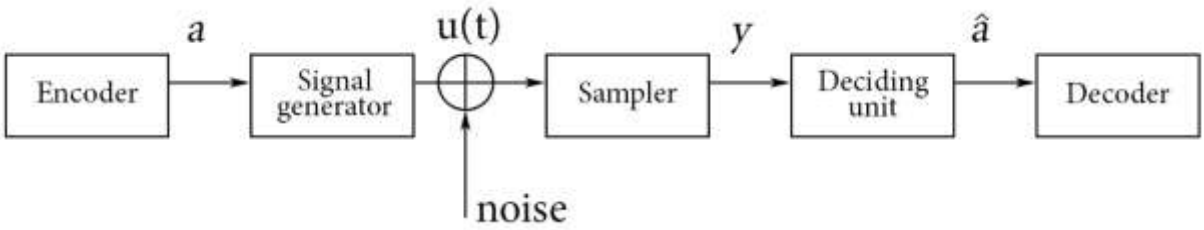


Figure 3.3: *The model of a noisy discrete channel*

In the figure above *a* is one of the channel symbols – "0" or "1" if the channel is binary – and $\hat{a}$ is its estimate at the recipient's side. The communication is time scheduled, the sampler takes a sample from the channel's u(t) physical signal (which is a continuous time value function of voltage) at the end of every t_0 time interval. The signal generator assigns a specific waveform to the channel symbol in the given time segment, based on *a*. For binary channels we need two base signals (a **base signal pair**).
Some base signal pairs:

- On-off keying, OOK. We assign a constant high voltage value to "1" and a low one to "0".
- Phase-shift keying, PSK. We assign a sine signal with 0 starting phase to "1", while "0" gets one with a $\pi/2$ starting phase.
- Frequency-shift keying, FSK. We assign a sine wave signal of lower frequency to "1" and a higher to "0".

The additive noise present in the channel distorts the waveform of the base signals. The decision maker has to identify $\hat{a}$ for a given time segment, based on a given point (e.g. from the middle) of the sample taken.

The Standard Decision Rule
The decision made is based on partitioning the range of the received y, where the A_i regions are identified for the corresponding a_i symbols (the partitioning is the rule itself). The decision rule is proper (for rational reasons), if

$$P(a_i|y) \geq P(a_j|y), \qquad \forall i,j \ \ where\ i \neq j\ and\ y \in A_i$$

The probability $P(a_i|y)$ is called a posteriori probability. Since we want to maximize it with our decision, the decision rule is called **maximum a posteriori** (MAP) or **standard** decision. The probability of a wrong decision:

$$P_{error} = P(y \in A_i, a \neq a_i)$$

It can be proven, that if we partition the range of y based on the standard decision rule, then the probability of false decision is minimal.
The decision problem is of **Bayes-type**, if

- the wrong decisions can be treated together, i.e. for $\hat{a} = a_i, a = a_j, i \neq j$ the cost of error is independent of *i* and *j* and
- there exists and can be given the **a priori** probability distribution *f(a)*.

If at least one of these conditions is not met, then the problem can be solved with general (statistical) hypothesis testing procedures. Such an example is the detection of enemy aircrafts and sounding the alarm: the cost of a false alarm and that of a missed one are not comparable.

The binary communication problem

Our task is to detect a channel symbol encoded via OOK (a_0 = "0" or a_1 = "1") sent over a binary symmetric channel that is affected by noise that has normal distribution. Based on the previously mentioned conditions, it is a Bayesian problem (it is the same, if we receive "1" instead of "0" or if we get "0" instead of "1"). To make our decision, first we modify the expression from the standard decision rule by inserting the conditional probabilities (in order to take into account the discrete distribution of the channel symbols and the continuous valued y):

$$f(a_i|y) = \frac{P(a_i)f(y|a_i)}{f(y)}$$

In this expression $P(a_i)$ is the a priori probability distribution, which is measured at the output of the encoder. $f(y|a_i)$ is the characteristics of the signal generator and the noise, which can be measured or calculated at the output of the receiver, when a continuous transmission of a_i is provided. To get our decision rule we need only to compare the numerators for the $i = 0$ and $i = 1$ cases, as the denominators are the same for every *i*. The intersection of curves $P(a_0)f(y|a_0)$ and $P(a_1)f(y|a_1)$ will give us the decision threshold (see Figure 3.4.) On the side where $P(a_0)f(y|a_0) < P(a_1)f(y|a_1)$ the decision will be "1", while on the other side it will be "0".

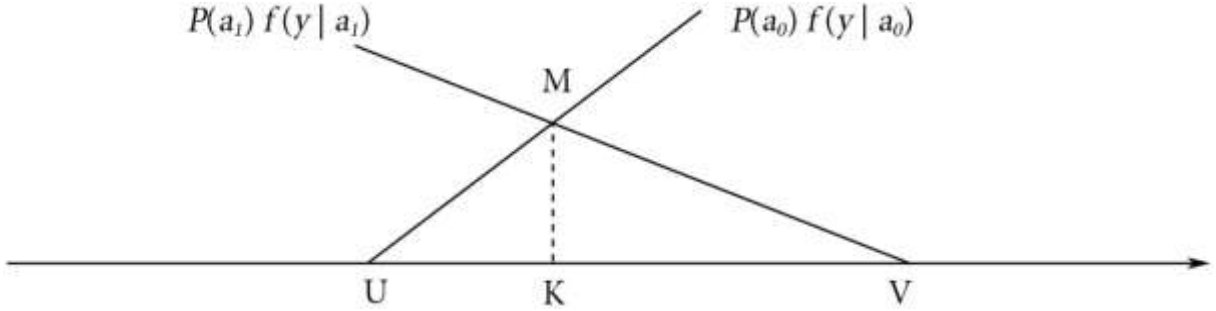


Figure 3.4: *The probability of error*

Limiting error probability, the erasure range

The p error probability of the channel is equal to the sum of the areas of triangles KMV and KMU in Figure 3.4. This error probability is an important parameter in channel design. If in our application this probability is higher then what we can accept, we can introduce an erasure range and thus modify our channel to act as a binary erasure channel (Figure 3.5).

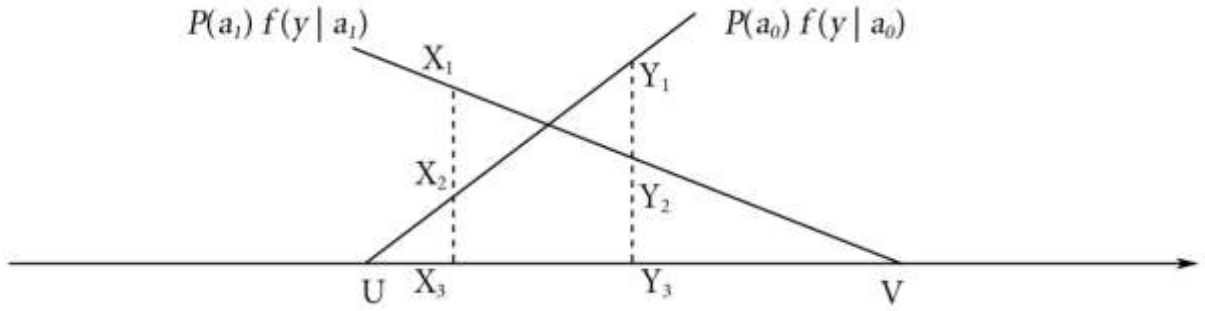


Figure 3.5: *Introducing the erasure range*

With this erasure range (between X_3 and Y_3) the probability of error is only the sum of the areas of triangles X_2X_3U and Y_2Y_3V. The deciding unit will detect an erasure symbol if y is between X_3 and Y_3. In such cases we need to resend the lost symbol (but this means extra cost due to channel reuse) or by using an error-correcting code we can fix the error on the recipient's side, if the number of erasures is less than the code length (these concepts will be detailed later on). The sum of the areas of trapezoids $X_1Y_2Y_3X_3$ and $X_2Y_1Y_3X_3$ is the probability of erasure error, which is also an important aspect in channel design.

The question is if we can find a method that **achieves maximum capacity usage** and provides **lossless** information transmission in general.

3.4. The Channel-Coding Theorem

The answer to the previous question is given by **Shannon's second Theorem** (a.k.a. Shannon's main theorem or the channel-coding theorem):

> Let us introduce redundant symbols in order to change a binary source's symbol blocks of length K to be of length N, and transmit the blocks through a noisy channel with capacity C. Then,
>
> - Then in case of K/N<C there *exists* an extension function (i.e. a channel encoding and decoding algorithm) where given a K large enough, the probability of block decoding error is smaller than any ε>0.
> - And in case of K/N>C **there is no** code that could achieve an arbitrary block decoding error probability.

What is a **block decoding error**? If the block size is increased due to redundant symbols (e.g. repeating the code words), then these "bigger" blocks are sent via the channel. This channel has a probability that it makes an error during the transmission of any "0" or "1" symbol (the output is different from what it should be). With the help of redundancy, we have a chance to recognize this error at the recipient's side, and to correct this error. Then $\hat{m} = m$. However, if we fail to notice the presence of the error, e.g. because the distortion moves the input into another valid code word, then we have no way of fixing this. And even if we notice and try to fix an error, it might happen that we end up with a valid code word that is different from the original. If **any of these** happen, then it is considered to be a block decoding error at the recipient's side.

Besides the theorem's **positive** statement

- it does not provide the channel coding method to use,
- decreasing ε results in a rapidly increasing block size K,
- lossless and errorless decoding is guaranteed only in general (not for specific cases)

The **negative** statement can be interpreted as "If we want to decrease the probability of block decoding errors, then we need to decrease the "density" of the information to be sent: by introducing redundant symbols in the blocks, we have to "dilute" it until it is below the channel capacity". And in case we *cannot* limit the presence of block decoding errors, then *we have no way of ensuring the usability of our communication system, **not even in the general case***.

Entropy rate

We introduce entropy rate R (a.k.a. code rate) as

$$R = \lim_{n\to\infty} \frac{H(c_0, c_1, \dots, c_{n-1})}{n}$$

where $H(c_0, c_1, \dots, c_{n-1})$ is the entropy of the n-times extended source C (on source C we mean the channel symbols sent to the channel, i.e. the output of the channel encoder. The 0, 1, … n-1 indexes represent a time sequence). In accordance with Section 2.3, $R \leq H(C)$, and equality means that C is a memoryless source: the consecutive symbols are statistically independent. Thus R is the measure of *the average information (bits) per channel use*.

We can state the channel coding theorem using the entropy rate, if we substitute K/N with R. Thus the theorem is applicable not just for block codes. In the special case of block codes, every block of length N will convey just the same amount of information as the message block of length K. That is because the extension does not add any extra information. In case of an optimally compressed source, the message block conveys at most K bit information. As this situation is most vulnerable when transmitting through a noisy channel, in channel coding we

suppose that at the encoder's input this K bit is the information content of a block of length K. This way, for the N symbols sent to the channel, the average information is K bit, while for one symbol the average information (entropy rate R) is K/N.

Example 3.4
Channel coding using parity bit 2 + 1, the 3 cases of Example 1.1
Code parameters: (3, 2)
The code: (for odd number of 1s the redundant part is 1, otherwise it is 0)

m_i	c_i
00	000
01	011
10	101
11	110

It can be noticed that the extension did not increase the entropy:

$$H(C) = H(M) = 2 \text{ bits}, \quad R = 2/3 \text{ bits}$$

3.5. Error Detection and Correction

In the decoder we try to detect and correct errors with the help of the redundancy that we introduced into the code word of length N. This redundancy means that of 2^N possible code words only 2^K are valid, because the original encoded message blocks are of length K. Naturally, the decoder knows the valid code words, and when the received word is invalid, it tries to find and forward the "most probable" legal code word. If it selects the correct one, then the **error correction** is successful.

It is also possible that when an invalid code word is found, the decoder does not try to correct it, instead it **notifies** the sender and requests a **resend** until a valid code word is received. However, by retransmitting the same code word over and over again, the efficiency of the channel (entropy rate) will decrease accordingly.

In some cases, it might occur that the channel transforms the input code word into another valid code word. When this happens, there is no way of detecting or correcting this error. This is why $\varepsilon = 0$ is not achievable in the channel coding theorem.

To help us in selecting the "correct" word, we introduce the **Hamming distance** d_H which is the number of bit positions that are different for two code words.

Example 3.5
For code words $c_1 = [011010]$ *and* $c_2 = [010110]$ $d_H(c_1, c_2) = 2$

Hamming distance is a **metric,** as for a and b binary code words of length N:

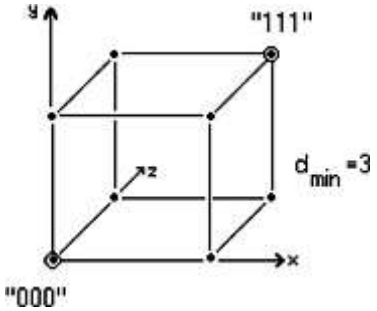
1. if $d_H(a, b) = 0$, then $a = b$
2. $d_H(a, b) = d_H(b, a)$
3. $d_H(a, c) \leq d_H(a, b) + d_H(b, c)$

Suppose that when correcting, the decoder selects the valid code word which is **closest** to the received word, and if there are more than one candidates, it selects from them randomly. It can be shown that this strategy selects the code word which is the most probable from the source statistics aspect (maximum likelihood decision), if the following two conditions hold:

1. The BSC used has an error probability $p < 0.5$
2. At the channel's input every code word (i.e. every message of length K) has the same probability of occurrence.

We can suppose the latter, as our main assumption in channel coding was that the source is already near-optimally encoded (compressed).

Example 3.6
*The "**Hamming cube"** for demonstrating error correction*



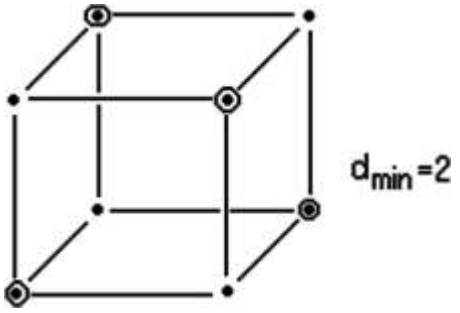
The cube shows a (3, 1) *code, where from the 8 possible code words only 2 are valid. We can correct errors most successfully if these 2 words are on the ends of a space diagonal (circled vertices). For any invalid code word, the closest valid code word can be found uniquely, which will also be the corrected word.*

*If we select the code shown above, we get the so-called **(3, 1) repetition code**.*

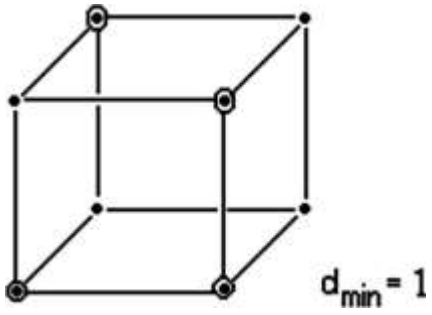
The Compromise of Correction and Detection

Our ability of correcting and/or detecting errors depends greatly on the **code distance** d_{min}, which is the minimum Hamming distance of the code. It can be noticed (or seen on the Hamming cube) that the bigger this distance, the more errors we can correct.

Example 3.7
For the code set of code words [0 1 1 1], [1 0 0 0], [0 1 1 0], [1 0 1 1] d_{min}=1



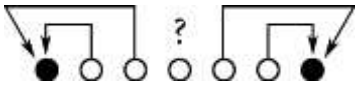
Parity code of Example 3.4



and a less efficient code

Denote the number of errors to be corrected as t_c. It is easy to see that in this case every code word should be of at least $2t_c$ distance from all other words ($d_{min} \geq 2t_c + 1$), otherwise our correction will be wrong. Its importance can be seen in Example 3.8, where empty circles represent invalid words and the filled are the valid ones. In the "middle" we do not know which code word should be chosen.

Example 3.8
Error correction when $d_{min} = 6$:



It can also be noticed that for *detection only*, a distance $d_{min} \geq 2t_d + 1$ is required. In Example 3.8 if we correct 2 bit errors, then for the middle one – when the distance is 3 for both words – we only detect the error. So we say we detect 3 errors. Without correction we could detect 5 errors, and 6 errors would result in a valid code word.

In most cases, we correct until a given number of errors is present, beyond that we just detect them. Of course, this is true only when $t_c < t_d$. But if we correct an error, then we cannot also detect it at the same time: for example, if we correct one error in the example above, we can detect only 4 errors. For a code word with 5 errors, we will "repair" it incorrectly into another valid word. So the relations are:

$$\begin{aligned} \textit{correction:} \quad & d_{min} \geq 2t_c + 1 \\ \textit{detection:} \quad & d_{min} \geq t_d + 1 \\ \textit{correction + detection:} \quad & d_{min} \geq t_c + t_d + 1 \textit{ and } d_{min} \geq 2t_c + 1 \end{aligned}$$

Example 3.9
Let $d_{min} = 6$. What are the code correction and detection possibilities?
Detect 5 errors OR
Correct 1 error *and* detect 4 OR
Correct 2 errors *and* detect 3

If our channel is a binary erasure channel, as the one in Example 3.3, and our received code word contains t_{erase} number of **erasure** symbols, i.e. we have erasure errors (but no other), then we can definitely find the original code word, if $d_{min} \geq t_{erase} + 1$ is satisfied, as we know the "positions" of these errors.

Based on which strategy we choose we end up with different probabilities of correcting into the wrong code word, i.e. executing a block decoding error at the recipient's side. Of course, the exact number of errors in a code word have a notable impact on this probability. From the user's point of view this probability is one of the most important properties of our

coding method. It can be seen that the expected value of the **number of errors**, for a BSC of error probability p and with block size N, is N · p, while the **probability of exactly t errors occurring** is:

$$P(t,p,N) = \binom{N}{t} p^t (1-p)^{N-t}$$

In practice, we usually do not correct all errors that the code could correct. If the distance of a code word from a valid code word is too high, we have it resent and we only correct code words that are close to a valid one. As a result, we get a slight decrease in the entropy rate due to the resending, but a rather significant decrease in the block decoding error probability.

3.6. Filling the Code Space

The Singleton Bound

We attach redundant symbols to a block to increase code distance. In the optimal case every additional symbol increases it with one. For example, for the left cube in Example 3.7 the parity bit increased the code distance to 2, while for the right one, the extension has no useful result.

Denote the number of redundant symbols with r=N-K. Then

$$d_{min} \leq r + 1$$

Rearranging the expression and exponentiating both sides with a base 2, we get the **Singleton bound**:

$$2^K \leq 2^{N-d_{min}+1}$$

where on the left side we get the maximal number of possible code words for a given d_{min}. When equality is achieved, i.e. all redundant symbols successfully increased the code distance, we have created a **maximum distance separable** (MDS) code.

Example 3.11
The (N, 1) *repetition code and the* (K+1, K) *parity code are MDS codes.*
Explanation: for the repetition code it is trivial. For the parity code, we only need to show that if the distance between two messages (both of length K) is 1, then the inserted parity bits will also be different, thus the distance will be 2. This is true, because if the distance is 1, it means that one of the words contains exactly one less "1" than the other, so if it has a parity bit "1", then the other gets "0" (and vice-versa).

The Sphere Packing Bound

The basic concept of correction and detection is that we have an N dimensional space, where we have our valid code words (as vectors), and for each we have a sphere with radius t_c (suppose that they do not overlap). Any (invalid) word inside a sphere is corrected into the valid code word in the centre. The way these spheres fill the N dimensional space shows us how "cost-efficient" our code is.

As an error can occur on any bit position, the number of invalid words with t errors for a given valid code word is

$$\binom{N}{t}$$

The number of code words within a sphere of radius t_c (including the valid word):

$$\sum_{i=0}^{t_c} \binom{N}{i}$$

If we sum all points inside the spheres of the possible 2^K valid code words, then this sum cannot exceed the total number of code words. This way we get the **Hamming bound** (a.k.a. **sphere packing bound**):

$$\sum_{i=0}^{t_c} \binom{N}{i} \leq 2^{N-K}$$

When the equality is satisfied – the spheres fill the space completely– then we have a **perfect code**. Such known perfect codes are repetition codes, Hamming codes (see Section 4.4) and the Golay code (further information in the corresponding literature). The corresponding values of N and K:

$t_c = 1$ ($d_{min} = 3$)	$t_c = 2$ ($d_{min} = 5$)	code type
(3, 1)	(5, 1)	repetition code
(7, 4)		Hamming code
(15, 11)		Hamming code
…		

The fact that parameters N and K satisfy the above mentioned equation with equality, does not necessarily mean that there exists a code with t_c code distance that could be constructed. For example, for $t_c = 2$ a (90, 78) code would seem acceptable, however, it can be proved that such code (for which $t_c = 2$ would require $d_{min} \geq 5$) does not exist.

Up to this point, we have introduced two simple channel codes, the parity and the repetition codes. Yet, the small block sizes of repetition codes and the small number of redundant symbols in parity codes mean that they have a limited significance in practical applications.